

范式转变：从传统编程到AI协作

环境配置好了，工具也能用了，但在真正开始写提示词之前，我们需要先回答一个问题：AI驱动的开发和传统编程到底有什么本质区别？

如果你曾经参与过传统的操作系统实验，应该很熟悉那套流程：阅读指导书，理解算法，在挖空的地方补充几行到几十行代码，编译，调试，直到测试通过。这个过程训练的是"实现能力"——你需要把算法描述翻译成能运行的代码。

但现在不一样了。

从"怎么做"到"做什么"

大型语言模型的出现，让编程的重心发生了根本性的转移。以前你需要关心"如何实现物理内存探测和页描述符初始化"，现在你更需要关心的是"page_init函数需要满足什么样的行为规格"。AI可以帮你写代码，但它需要你需求说清楚。

代码实现依然重要。实际上，正因为AI能生成代码，你反而更需要理解代码背后的系统语义。你要知道设备树提供的物理内存范围如何转换为可管理页面，OpenSBI、内核镜像和Page描述符数组占用的空间如何排除，以及页面边界如何对齐。只有理解这些，你才能写出让AI生成正确代码的提示词。

有人可能觉得"让AI写代码"是在偷懒。但设计一个清晰、完整、无歧义的需求，往往比写代码本身更难。在传统实验里，指导书已经把算法步骤、数据结构、甚至伪代码都给你了，你要做的是"照着翻译"。但现在，你需要从功能需求出发，自己梳理出：这个模块负责什么，不负责什么；需要哪些前置条件，保证哪些后置状态；等等。角色变了，从"实现者"变成了"设计者"。

能力的重新定义

现在你需要的能力不再只是"会写代码"，而是：

需求分析能力。把模糊的功能描述转化为精确且规范的格式。"实现物理内存初始化"不是一个好需求，"实现page_init，根据设备树信息设置va_pa_offset和npa，建立Page描述符数组，排除OpenSBI、内核镜像和描述符数组占用的空间，并将对齐后的完整空闲页面交给当前pmm_manager管理"才是。

系统思维能力。操作系统不是一堆孤立的函数，而是一个复杂的集合。理解各个模块之间的依赖关系，知道修改一个地方会影响哪些其他地方。为什么必须先根据设备树确定物理内存范围，再计算Page描述符数组占用的物理空间并注册空闲区？因为只有排除已经被OpenSBI、内核镜像和描述符数组占用的页面，物理页分配器才不会把正在使用的内存交给后续分配。

验证能力。AI生成的代码不一定对。设计测试用例，理解每一条需求对应的验收标准，知道如何通过测试来验证代码的正确性。

迭代能力。第一版提示词很少能一次成功。根据编译错误、测试失败、运行结果来判断是提示词哪里说得不够清楚，然后进行迭代优化。

但本质没变

不管用什么工具，不管用什么方式，操作系统的核心概念没有变。物理内存探测还是物理内存探测，Page描述符布局还是Page描述符布局，虚拟地址和物理地址之间的转换关系还是那个转换关系。AI不会帮你理解这些概念，它只是一个工具，帮你更高效地把理解转化为代码。

所以，如果你发现自己不理解某个概念，不要指望提示词能掩盖这个问题。相反，你会发现，当你试图把一个概念写成提示词时，如果理解不到位，你根本写不清楚。这反而是一种更好的学习方式：通过“向AI描述需求”这个过程，倒逼自己把概念理解透彻。

传统编程就像你是建筑工人，拿着图纸，一砖一瓦地砌墙。AI驱动编程更像你是建筑师，需要设计图纸，标注每堵墙的位置、高度、材料、承重要求。AI是施工队，按照你的图纸施工。图纸画得不清楚，施工队就会建出四不像。图纸画得精确，施工队才能建出符合要求的建筑。而要画好图纸，你不仅要懂建筑，还要懂得如何把脑海中的构想转化为可执行的规格说明。

这个实验要训练的就是这种能力。

下一步：学会画图纸

角色转变之后，接下来的问题是：如何把需求写清楚？如何让 AI 准确理解你的意图？

这就是提示词工程要解决的问题。接下来会详细介绍提示词的写法、结构和技巧，以及如何在操作系统开发中应用这些原理。